



نظري

9



9



1080 sp

كلية الهندسة المعلوماتية

السنة الثالثة

PHP RESTFUL Web Service API

د. غاندي هسام

Programming Languages



RB Informatics; 26/12/2025

لغات البرمجة

RESTful

#REST stands for Representational State Transfer and it is an architectural style that enables the communication between systems

يرمز REST إلى نقل الحالة التمثيلية، وهو نمط معماري يُمكن من التواصل بين الأنظمة.

#The REST concept is defined by specific rules, constraints, or principles. A system, application, service, or anything else that adheres to these REST principles is called RESTful.

يُعرف مفهوم ريسـت بقواعد أو قيود أو مبادئ محددة. ويُطلق على أي نظام أو تطبيق أو خدمة أو أي شيء آخر يلتزم بمبادئ ريسـت REST.

#Web services that follow the RESTful principles are RESTful services. The URI is used to access RESTful services to get the resources

تسمى خدمات الويب التي تتبع مبادئ RESTful بخدمات RESTful

يُستخدم معرف الموارد الموحد URI للوصول إلى خدمات RESTful والحصول على موارد.

#In the RESTful glossary, the resources are nothing but the data and functions. So eventually we will call the web services via URI to access functions and thereby get the resource data

في مصطلحات RESTful:

لا تمثل الموارد سوى البيانات والتوابع، لذا سنستدعي خدمات الويب عبر URL للوصول إلى التوابع ومن ثم الحصول على بيانات المورد

Use of REST API استخداماته:

1. An API (Application Programming Interface) is a rule set that an application follows to transfer data

واجهة برمجة التطبيقات (API) : هي عبارة عن مجموعة من القواعد التي يتبعها التطبيق لنقل البيانات.

2. here are multiple APIs for PHP applications, some of them are REST API, SOAP API, GraphQL API, XML-RPC API, and JSON-RPC API.

توجد واجهات برمجة تطبيقات متعددة لتطبيقات PHP بعضها عبارة عن واجهات برمجة تطبيقات

REST و SOAP و GraphQL و XML-RPC و JSON-RPC

3. The process to set up each API in PHP is different because of the different architecture and use cases

تختلف عملية إعداد كل واجهة برمجة تطبيقات (API) في لغة PHP بسبب اختلاف البنية وحالات الاستخدام

- The reasons why REST API is popular in PHP are as follows:

فيما يلي الأسباب التي تجعل واجهة برمجة التطبيقات REST شائعة في لغة PHP

○ متوافق مع جميع خدمات الاستضافة

Compatible with all hosting services

○ يمكنك التكامل بسهولة مع قواعد البيانات مثل MySQL

Effortlessly integrate with databases like MySQL

○ استخدم أساليب HTTP الشائعة مثل GET , POST , DELETE , PUT لتنفيذ المهام

Use common HTTP methods such as GET, POST, PUT, and DELETE to carry out

Definition and Principles of REST API

- **Statelessness :**

Each request from a client to a server must contain all the information the server needs to fulfill that request. The server does not store any information about the client's state between requests

يجب أن يحتوي كل طلب من العميل إلى الخادم على جميع المعلومات التي يحتاجها الخادم لتلبية هذا الطلب. لا يقوم الخادم بتخزين أي معلومات حول حالة العميل بين الطلبات.

- **Client-Server Architecture العميل – الخادم**

يُعدّ كل من العميل والخادم كيانين منفصلين يسمح هذا الفصل بين المهام للعميل والخادم بالتطور بشكل مستقل.

- **Cacheability إمكانية التخزين المؤقت**

Responses from the server can be cached by the client or intermediate proxies to improve performance

يمكن للعميل أو الخوادم الوكيل الوسيطة تخزين الردود الواردة من الخادم مؤقتًا لتحسين الأداء.

- **Uniform Interface: واجهة موحدة**

REST APIs have a consistent and uniform interface, simplifying and decoupling the architecture. This uniformity allows different clients to interact with the server in a consistent manner.

تتميز واجهات برمجة التطبيقات من نوع REST بواجهة متسقة وموحدة مما يبسط بنية النظام ويفصل مكوناته يتيح هذا التوحيد للعملاء المختلفين التفاعل مع الخادم بطريقة متناسقة.

:Advantages of REST API in PHP

❖ مميزات REST API in php

- 1. Simplicity and Flexibility:** PHP's syntax is simple and easy to learn, making it an excellent choice for both beginners and experienced developers.
البساطة والمرونة: تتميز لغة PHP ببنية بسيطة وسهلة التعلم، مما يجعلها خياراً ممتازاً لكل من المبتدئين والمطورين ذوي الخبرة.
- 2. Seamless Database Integration:** PHP provides robust support for connecting to various databases, including MySQL, PostgreSQL, and SQLite.
تكامل سلس لقواعد البيانات: توفر لغة بي اش بي دعماً قوياً للاتصال بقواعد بيانات متنوعة بما في ذلك (المذكورة في السلايدات)
- 3. Compatibility with Client-Side Applications:** PHP-based REST APIs can easily communicate with client-side applications written in various languages such as JavaScript, Python, and Java.
التوافق مع تطبيقات جانب العميل: يمكن لواجهات التطبيقات (ريست أي بي أي) المبنية على لغة التواصل بسهولة مع تطبيقات جانب العميل المكتوبة بلغة برمجة مختلفة مثل جافا سكريبت وبايثون وجافا.
- 4. Wide Range of Libraries and Frameworks:** PHP has an extensive collection of libraries and frameworks, such as Laravel and Symfony, which simplify the process of building RESTful APIs.
مجموعة واسعة من المكتبات والأطر البرمجية: تحتوي على مجموعة واسعة من المكتبات وأطر من العمل، مثل لارافيل، سيمفوني، والتي تبسط عملية بناء واجهات برمجة التطبيقات من نوع

:Key Concepts of RESTful APIs

❖ المفاهيم الأساسية لواجهات برمجة التطبيقات RESTful

- **Resources:** In a RESTful API, resources are the key entities that clients interact with. These can be anything from user data to products or services

مصادر: في واجهة برمجة التطبيقات من نوع, تعد الموارد هي الكيانات الرئيسية التي تتفاعل معها تطبيقات العملاء. يمكن أن تكون هذه الموارد أي شيء, بدءاً من بيانات المستخدمين وصولاً إلى منتجات أو الخدمات.

- **HTTP Methods:** RESTful APIs utilize standard HTTP methods:

RESTful تستخدم واجهات برمجة التطبيقات من نوع

HTTP أساليب القياسية

-> **GET:** Retrieve data from the server استرجاع البيانات من الخادم

-> **POST:** Create a new resource on the server إنشاء مورد جديد على الخادم

-> **PUT:** Update an existing resource تحديث مورد موجود

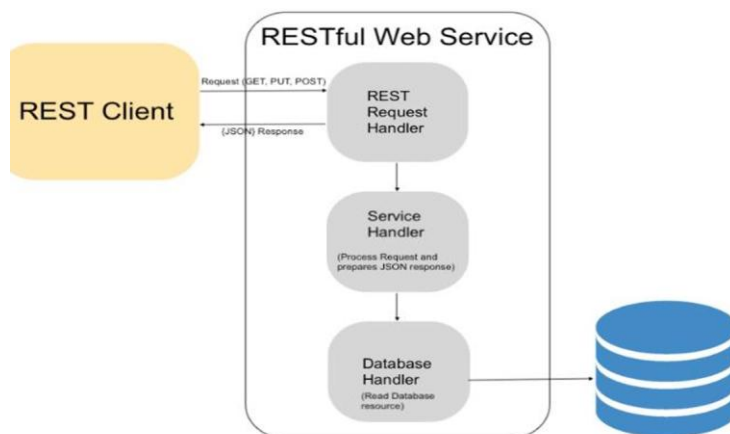
-> **PATCH:** Apply partial modifications to a resource تطبيق تعديلات جزئية على مورد معين

-> **DELETE:** Remove a resource from the server حذف مورد من الخادم

- **Endpoints:** Each resource is accessed through a specific URL, known as an endpoint.

نقاط الوصول: يتم الوصول إلى كل مورد من خلال عنوان

URL محدد, يُعرف باسم نقطة الوصول



Dr. Ghadi Hssam

7

RESTful web services vs RPC web services

S.No.	REST API	RPC API
1	REST is resource-oriented.	RPC is action-oriented.
2	Supports HTTP methods GET, POST, PUT, PATCH, and DELETE.	RPC only supports GET and POST requests.
3	It provides flexibility for hardware architecture	no flexibility in RPC for hardware architecture
4	It supports hypermedia and hyperlinks	It does not supports hypermedia and hyperlinks
5	REST allows to specify Content-types or accept headers	Require payloads of a few data types as XML for XML-RPC.

Building Your First REST API in PHP

1. Creating a PHP Project Directory:

Create a new directory for your project. Inside this directory, create folders for models, views, and controllers to follow the MVC pattern

إنشاء دليل مشروع: أنشئ مجلدًا جديدًا لمشروعك. داخل هذا المجلد، أنشئ مجلدات للنماذج والعروض MVC ووحدات التحكم لاتباع نمط

2. Structuring the Project: هيكلية المشروع

Models: These represent the data structure. For example, a User model to handle user data

النموذج: تمثل هذه العناصر بنية المعطيات، على سبيل المثال، نموذج المستخدم لإدارة بيانات المستخدمين

Views: These handle the presentation layer. In REST APIs, views are often represented by JSON responses

تتولى هذه الطبقة عرض المحتوى. في واجهات برمجة تطبيقات REST غالبًا ما يتم تمثيل طرق العرض بواسطة استجابات JSON.

Controllers: These handle the business logic and interact with models to fetch or update data.

وحدات التحكم: تتولى هذه الوحدات معالجة منطق الأعمال والتفاعل مع النماذج لجلب البيانات أو تحديثها

3. Writing the Basic PHP Script for REST API:

Create an index.php file as the entry point for your API. Implement a basic routing mechanism to handle different HTTP requests (GET, POST, PUT, DELETE)

كتابة صفحة PHP أساسية لواجهة برمجة تطبيقات REST: أنشئ ملف index.php ليكون بمثابة نقطة الدخول لواجهة برمجة التطبيقات API نفذ آلية توجيه أساسية للتعامل مع طلبات HTTP المختلفة

Step 1: Designing Your API

❖ الخطوة الأولى: تصميم واجهة برمجة التطبيقات API الخاصة بك

Before writing any code, it's essential to outline the resources you intend to expose through your API and the operations that clients can perform on those resources

قبل كتابة أي كود، من الضروري تحديد الموارد التي تنوي عرضها من خلال واجهة برمجة التطبيقات (API)



والعمليات التي يمكن للعملاء تنفيذها على تلك الموارد

The following methods are standard in RESTful API design:

تعتبر الطرق التالية قياسية في تصميم واجهات برمجة التطبيقات

RESTful:

GET: This method retrieves data from the server. It's commonly used to fetch a list of resources or a specific resource based on an identifier.

تسترجع هذه الطريقة البيانات من الخادم. وهي شائعة الاستخدام لجلب قائمة من الموارد أو مورد محدد بناءً على معرف.

مثال:

GET /api/v1/posts: Retrieve all blog posts

GET /api/v1/posts/{id}: Retrieve a specific blog post by its ID

POST: This method is used to create new resources on the server. When a client sends a POST request, it typically includes a request body containing the data for the new resource. تستخدم هذه الطريقة لإنشاء موارد جديدة على الخادم عندما يرسل العميل طلب post فإنه عادةً ما يتضمن نص الطلب الذي يحتوي على بيانات المورد الجديد

مثال:

POST /api/v1/posts: Create a new blog post

PATCH: This method is used to apply partial modifications to a resource. Unlike PUT, which replaces the entire resource, PATCH allows clients to send only the fields that need to be updated

تستخدم هذه الطريقة لتطبيق تعديلات جزئية على مورد على عكس PUT ، الذي يستبدل المورد بأكمله، تسمح PATCH للعملاء بإرسال الحقول التي تحتاج إلى تحديث فقط.

مثال:

PATCH /api/v1/posts/{id}: Update specific fields of an existing blog post

تحديث حقول محددة في منشور مدونة موجود

PUT: This method updates an existing resource or creates a new resource if it does not exist. When using PUT, the client sends a complete representation of the resource. Example تستخدم هذه الطريقة لتحديث مورد موجود أو إنشاء مورد جديد إذا لم يكن موجوداً. عند استخدام put يرسل العميل تمثيلاً كاملاً للمورد

مثال:

PUT /api/v1/posts/{id}: Update the entire blog post with the specified ID

تحديث منشور المدونة بالكامل باستخدام الرقم المحدد

DELETE: This method removes a resource from the server. It is used to delete a specific resource identified by its URL

تزيل هذه الطريقة موردًا من الخادم. وتستخدم لحذف مورد مُحدد يتم التعرف عليه بواسطة عنوان الخاص به Url
مثال:

DELETE /api/v1/posts/{id}: Delete a blog post with the specified ID.

حذف منشور مدونة باستخدام الرقم المحدد.

Resource Planning تخطيط الموارد

In addition to defining the methods, you should plan the resources that your API will manage. For instance, if you are developing a blogging platform, you might have the following resources:

بالإضافة إلى تحديد الطرق, يجب عليك تخطيط الموارد التي ستديرها واجهة برمجة التطبيقات على سبيل المثال , اذا كنت تقوم بتطوير منصة تدوين, فقد تحتاج إلى الموارد التالية:

#Posts: Represents individual blog entries المنشورات: تمثل منشورات المدونة الفردية

#Comments: Represents user comments on blog posts التعليقات: تمثل تعليقات المستخدمين على منشورات المدونة

#Users: Represents authors or readers of the blog المستخدمين: يمثلون مؤلفي أو قراء المدونة

By clearly outlining your API's resources and their corresponding actions, you create a blueprint that guides the development process.

من خلال تحديد موارد واجهة برمجة التطبيقات (API) والإجراءات المقابلة لها بشكل واضح, فإنك تنشئ مخططاً يوجه عملية التطوير

Step 2: Creating the API Structure

❖ الخطوة الثانية: إنشاء بنية واجهة برمجة التطبيقات (API)

Create a folder structure for your API project. Here's a simple example:

أنشئ بنية مجلدات لمشروع واجهة برمجة التطبيقات (API) الخاص بك. إليك مثال بسيط:

```
/api
/config
  database.php
/controllers
  PostController.php
  CommentController.php
/models
  Post.php
  Comment.php
index.php
```

Configuring the Database **تهيئة قاعدة البيانات:**

In your config/database.php, establish a connection to your database:

في ملف config/database.php الخاص بك، قم بإنشاء اتصال بقاعدة البيانات الخاصة بك:

```
<?php
$host = 'localhost';
$db_name = 'your_database_name';
$username = 'your_username';
$password = 'your_password';

try {
    $conn = new PDO("mysql:host=$host;dbname=$db_name", $username, $password);
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}
?>
```

Step 3: Creating Models

#Models represent your data and contain functions to interact with the database. For example, in models/Post.php, you might have

إنشاء نموذج: النماذج تمثل بياناتك وتحتوي على التوابع للتعامل مع قاعدة البيانات، على سبيل المثال في models/Post.php قد يكون لديك

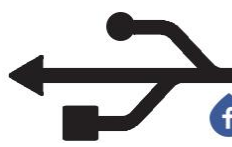
```
class Post {
    private $conn;

    public function __construct($db) {
        $this->conn = $db;
    }

    public function getAllPosts() {
        $stmt = $this->conn->prepare("SELECT * FROM posts");
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }
}
```

returns an array indexed by column name as returned in your result set

Step 4: Creating Controllers



Controllers handle the incoming requests and interact with the models. In controllers/PostController.php, you can set up your endpoints:

تتولى وحدات التحكم معالجة الطلبات الواردة والتفاعل مع النماذج في ملف يمكنك إعداد نقاط النهاية الخاصة بك controllers/PostController.php

```
require_once 'controllers/PostController.php';

$requestUri = explode('/', trim($_SERVER['REQUEST_URI'], '/'));

if ($requestUri[1] === 'posts') {
    require 'controllers/PostController.php';
} else {
    http_response_code(404);
    echo json_encode(['message' => 'Resource not found']);
}
```

Step 5: Routing Request

In index.php, you will route incoming requests to the appropriate controller based on the requested endpoint:

في ملف index.php ستقوم بتوجيه الطلبات الواردة إلى وحدة التحكم المناسبة بناءً على نقطة النهاية المطلوبة.

```
require_once 'controllers/PostController.php';

$requestUri = explode('/', trim($_SERVER['REQUEST_URI'], '/'));

if ($requestUri[1] === 'posts') {
    require 'controllers/PostController.php';
} else {
    http_response_code(404);
    echo json_encode(['message' => 'Resource not found']);
}
```

انتهت المحاضرة

<The end>